

The NAF Framework

A shared view for network automation

Six building blocks, one vocabulary



Bart Dorlandt

- Freelance Network Automation Solution Architect
- Lives by: ***There must be a better way***
- Organizer of NLNAM
- Co-organizer of PyUtrecht
- Advisory board member of Network Automation Forum (NAF)

The conversation we keep having

"We use Ansible, Nautobot, Grafana and a pile of Python."

That answers **what tools we bought/use.**

It does not answer:

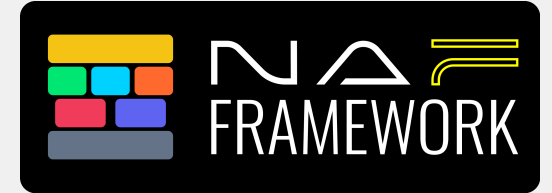
- What job does each tool actually do?
- What is missing?
- What is duplicated three times?

Starting from tools is backwards

- No consensus on what the tool should do
- Two tools own the same or overlapping data
- Nobody can say *where* the next feature belongs
- Onboarding = tribal knowledge transfer

We had no shared vocabulary to talk about the problem.

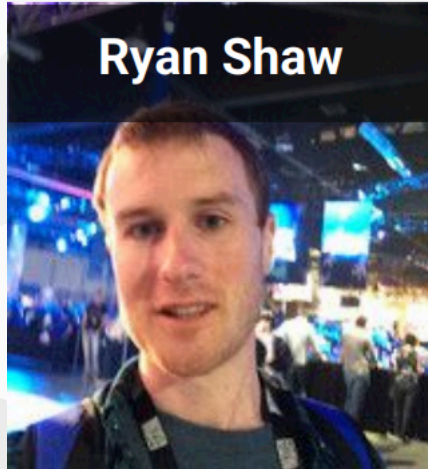
Enter the NAF Framework



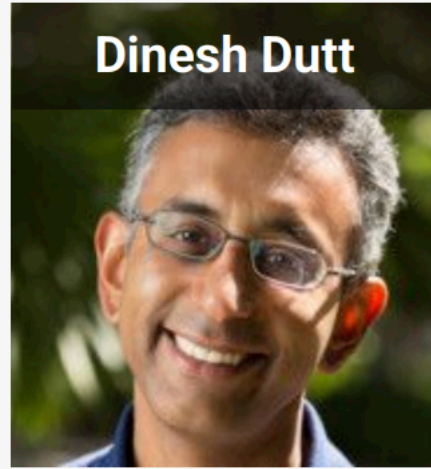
A **modular, vendor-neutral reference model** for network automation.

- Published by the Network Automation Forum
- Defines **functional building blocks**, not products
- Design around required functionality — *then* pick tools

The workgroup



Ryan Shaw



Dinesh Dutt



Christian Adell



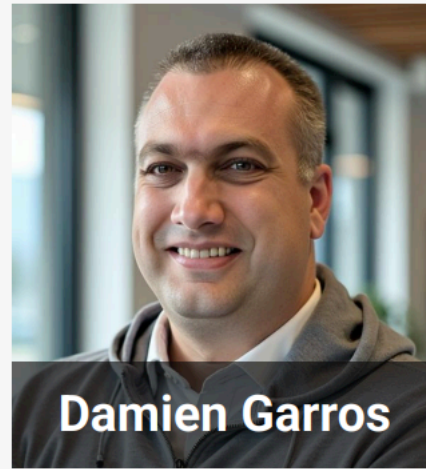
Claudia de Luna



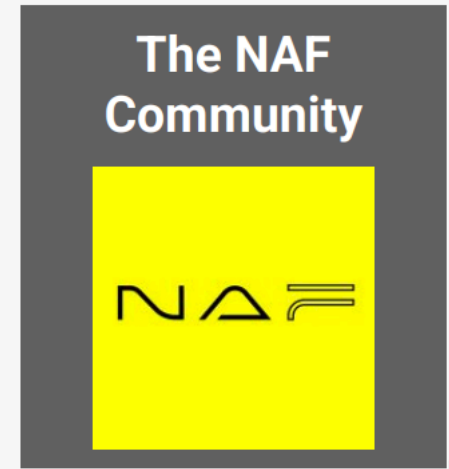
Srividya Iyer



Wim Henderickx



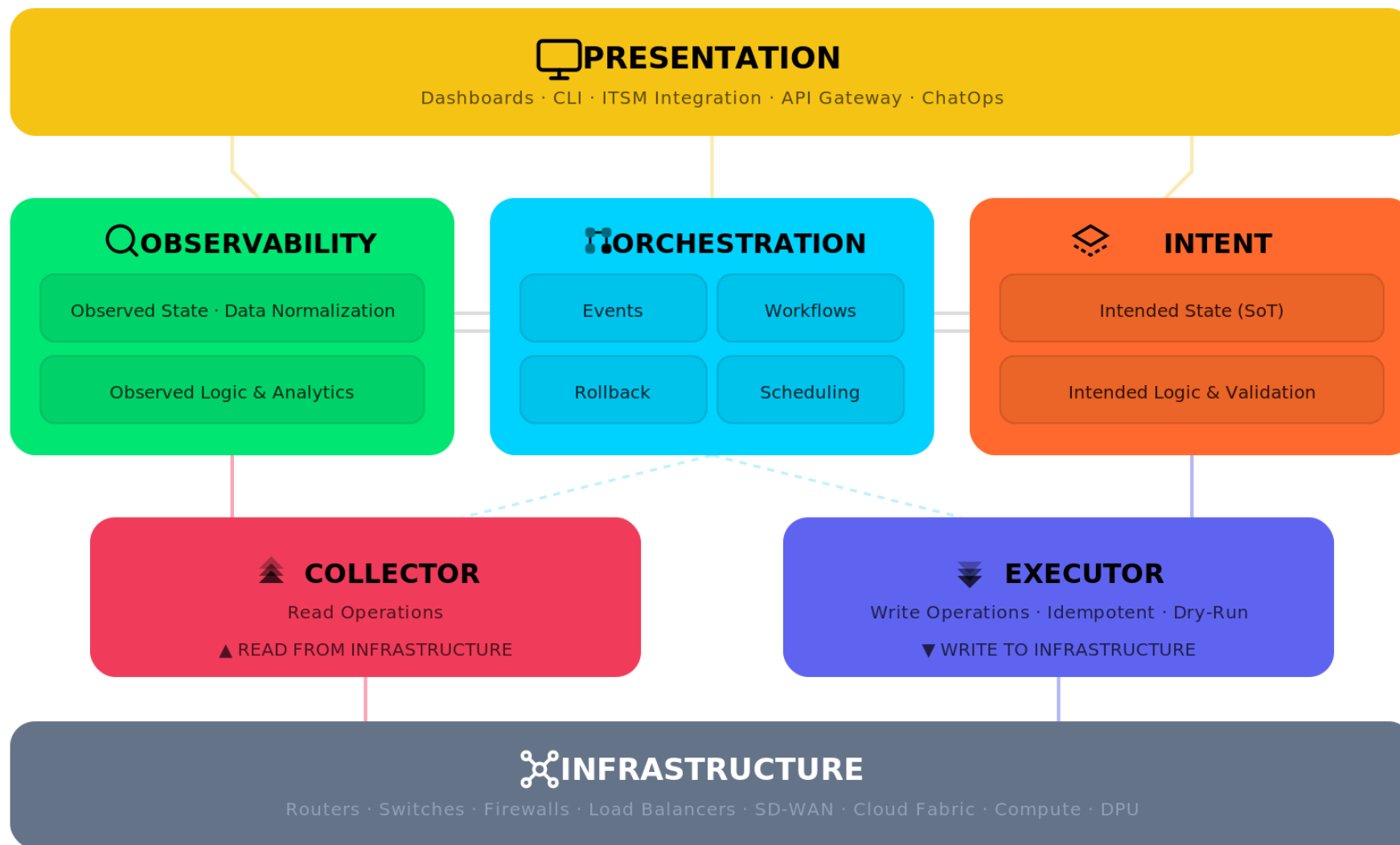
Damien Garros



**The NAF
Community**

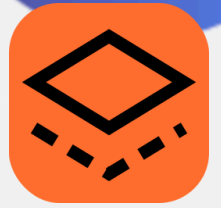
Network Automation Framework

REFERENCE ARCHITECTURE



NAF Reference Architecture

Intent



Defines the logic to handle and the persistence layer to store the desired state of the network, including both configuration and operational expectations

MUST

be capable of representing, in a structured form, any network-related aspect

MUST

support create, read, update, and delete operations

MUST

be exposed through a standardized, well-documented API

SHOULD

provide a consistent and unified view of the desired state

SHOULD

use a neutral representation that will be derived into vendor-specific configuration artifacts

SHOULD

include metadata that supports effective data governance

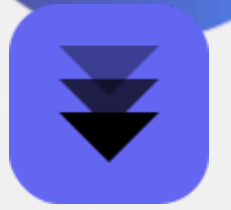
SHOULD

be transactional, and provide versioned access to data

MAY

include all the logic related to intended state management

Executor



Encompasses the actual tasks applied to the network to drive changes (e.g., updating configuration) as guided by the intended state.

MUST

be capable of interacting with any of the supported network write interfaces

SHOULD

come from the intended state or be derived from it

SHOULD

support any network operation that alters the network state

SHOULD

provide a dry-run operation to check the expected result of the execution

SHOULD

support transactional execution of the changes

MAY

support both imperative and declarative approaches

Collector

Focuses on retrieving (i.e., reading) the actual state of the network

MUST

include capabilities for retrieving live data from the network using read interfaces (push, pull)

Observability



Stores the actual network state, and defines the logic to process the observed data.

MUST

support historical data persistence

MUST

offer programmatic access to this data

SHOULD

offer a capable query language to extract the data

SHOULD

expose relevant insights into the current network state

SHOULD

automatically generate events when discrepancies are detected

SHOULD

normalize data into a vendor-agnostic data model

MAY

have events processed by humans or connected to the Orchestrator

MAY

be enriched with contextual information from the intended state

Orchestrator



Coordinates activities across various building blocks

MUST

enable coordination of processes across the various building blocks

SHOULD

follow an event-driven approach for process execution

SHOULD

provide a dry-run operation to check the expected result of the workflow

SHOULD

provide the ability to schedule the execution of a workflow

SHOULD

allow execution logic to perform reverse/compensating actions

SHOULD

provide logging and traceability of past and current workflows

MAY

include logic to correlate multiple events and infer relationships

Presentation



Provides the interfaces through which users interact with the system, including dashboards, graphical user interfaces

MUST

provide robust authentication and authorization capabilities

MAY

support both read and write interactions

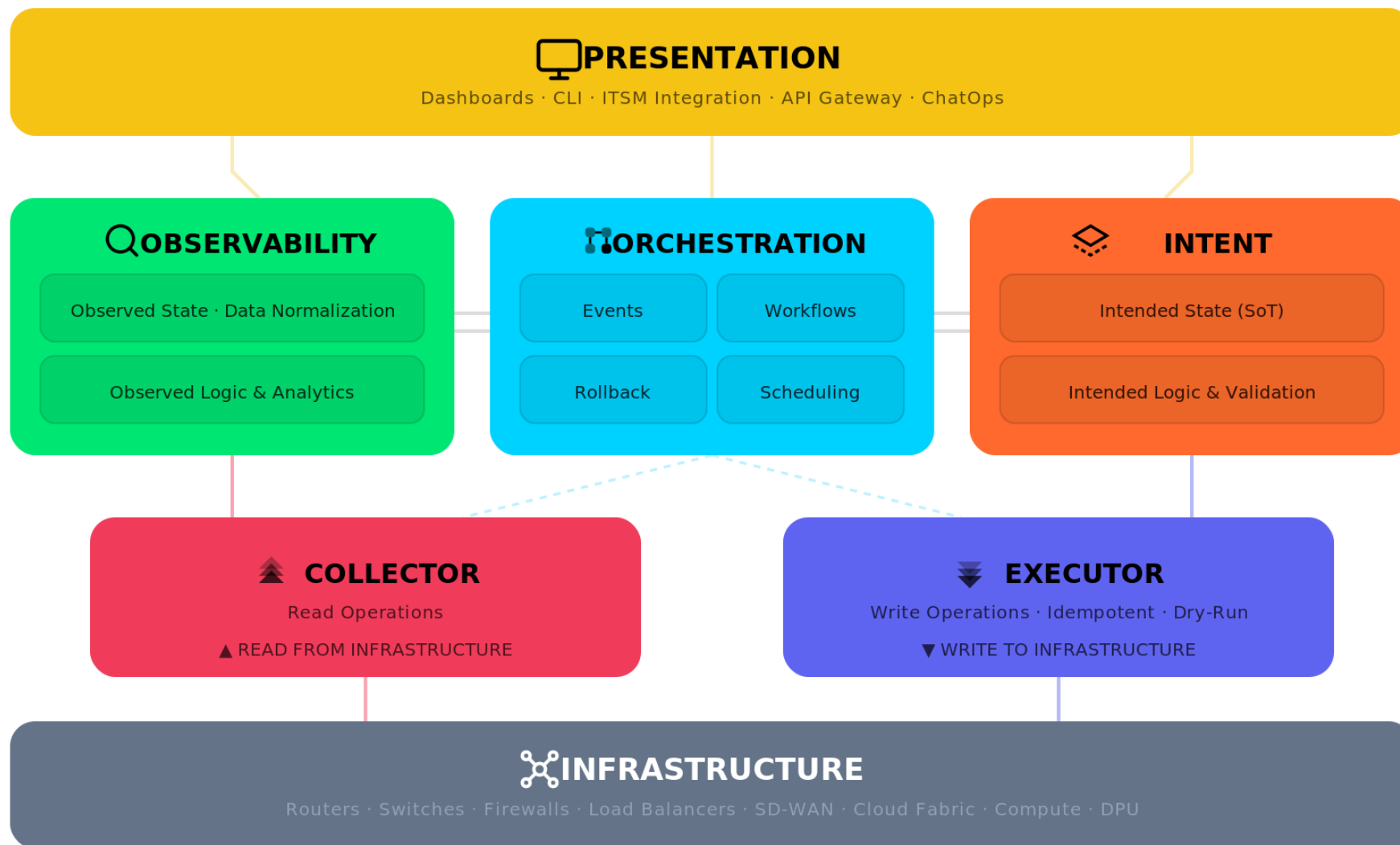
MAY

take various forms depending on the needs of the end user

*This does **not** imply a single pane of glass.*

Network Automation Framework

REFERENCE ARCHITECTURE



NAF Reference Architecture

The rules that make it work

- Blocks are **functions**, not products
- One block MAY be many components — or one tool MAY cover several blocks
- Everything exposes a machine-friendly, schema-documented API
- Version control and CI/CD for all of it
- Make operations idempotent and transactional where possible

How to actually use it

1. **Map** what you have onto the six blocks
2. Look for the **empty** boxes
3. Look for the **crowded** boxes — multiple tools writing the same data
4. Pick the one gap that hurts most; fill only that
5. Repeat

The map is not a shopping list.

What you get out of it

- A vocabulary your whole team shares
- A place to put every new requirement
- An honest picture of duplication and gaps
- Vendor conversations on *your* terms: "which block do you fill?"
- A way to start narrow without painting yourself into a corner

Call to Action

Take your own stack and ask:

1. **Where is my intent?** Is it a database or a set of habits?
2. **Can I see drift?** Actual state versus intended state, automatically?
3. **Who writes?** Does anything besides the Executor touch the devices?
4. **What is the one empty box?** Start there.

NAF Matrix - Christian Drefke

<https://naf-framework-solution-matrix.vercel.app/>

NAF **NAF Automation Framev**

Click a cell to toggle it · double-click a label to rename · drag the ≡ handles to reorder · click a layer switch to recolor

	PRESENTATION	INTENT	ORCHESTRATION	EXECUTOR	COLLECTOR	OBSERVABILITY
Infrahub Source of truth + UI	•	•	•	•	•	•
AAP Runbooks + web UI	•	•	•	•	•	•
Ansible Config push (NET...	•	•	•	•	•	•
gNMic Telemetry collection	•	•	•	•	•	•
Netbox IPAM + DCIM	•	•	•	•	•	•
Prometheus Metrics + alerts	•	•	•	•	•	•
Grafana Compliance dash...	•	•	•	•	•	•
SuzieQ State observability	•	•	•	•	•	•
Kentik Flow observability	•	•	•	•	•	•

Q&A

- Bart Dorlandt
- <https://linkedin.com/in/bartdorlandt/>
- <https://dreamnetworking.nl/>
- <https://net-auto.nl/>

The framework:

reference.networkautomation.forum/Framework/Framework

